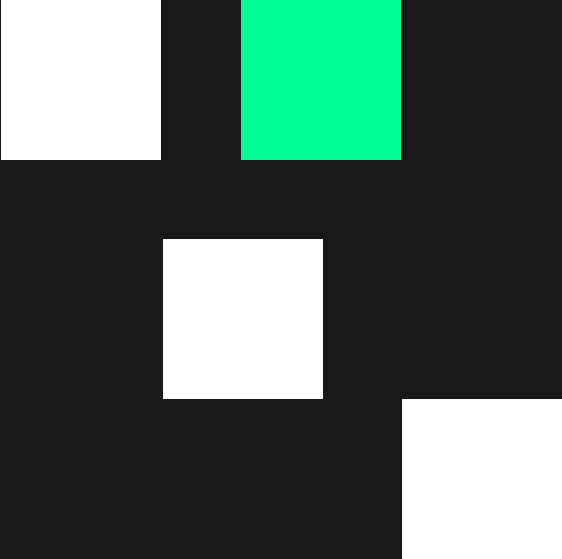


{EPITECH}

< G-CPP-500 · ADVANCED C++ />

KICK-OFF R-TYPE

G5 · Advanced C++



WE POWER TECH

{ <TECH TOGETHER TOMORROW> }

R-TYPE, LE JEU

LE JEU

- Jeu d'arcade spatial de 1987
- Shoot'em up à défilement horizontal

UN GAMEPLAY SIMPLE

- Tuer des aliens malicieux
- Récupérer des power-ups
- Vaincre les boss
- Passer au niveau suivant



OBJECTIFS

CONCEVOIR VOTRE MOTEUR DE JEU EN RÉSEAU EN C++

- Programmation réseau
- Architecture de moteur de jeu
- Jeu multijoueur en réseau

EXPLORER DES SUJETS AVANCÉS

- Architecture
- Réseau
- Game design

ADOPTER DE BONNES PRATIQUES DE GÉNIE LOGICIEL

- Documentation
- Outils de build et de packaging
- Workflow de VCS
- Programmation multiplateforme

OBJECTIFS

PARTIE 0 : GÉNIE LOGICIEL

- Un produit « professionnel »
- Évalué aux deux soutenances

PARTIE 1 : PROTOTYPE

- Fonctionnalités de base
- Gameplay basique
- Évaluée à la 1^{re} soutenance

PARTIE 2 : LOGICIEL AVANCÉ

- Améliorer le prototype
- Choisir ses sujets avancés
- Évaluée à la 2^e soutenance

+ < 1^{re} soutenance /> Ce qu'évalue la 1^{re} soutenance ne sera pas réévalué à la 2^e.

Début du projet
Kick-off 1

1^{re} soutenance
Kick-off 2

Soutenance finale

3 semaines

3 semaines

PARTIE 0 : GÉNIE LOGICIEL ET DOCUMENTATION

PARTIE 1 : ARCHITECTURE ET PREMIER PROTOTYPE

PARTIE 2 : SUJETS AVANCÉS

TOUT COMPTE_

À CHAQUE SOUTENANCE

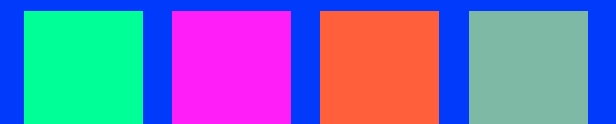
**TOUT CE QUE VOUS FAITES EST VALORISÉ, À
CONDITION DE LE MONTRER AUX
ÉVALUATEURS**

PAR EXEMPLE

- Outils internes, éditeurs, vues de debug
- L'outillage IA que vous construisez (skills, agents, commandes) pour la production, la documentation, le contrôle qualité, les tests fonctionnels
- Tests, benchmarks, profiling
- Tout ce qui va au-delà du sujet



PARTIE 0 : GÉNIE LOGICIEL



PROJET => PRODUIT

PROJET

« Ça marche sur ma machine »

« Il te faut ces dépendances »

« Ping-moi si t'as des questions »

« Envoie un message sur Discord si t'as des idées »



PRODUIT

ÇA MARCHE PARTOUT



LES DÉPENDANCES S'INSTALLENT SEULES



UNE DOCUMENTATION SOIGNÉE



DES RÈGLES DE CONTRIBUTION

MAÎTRISEZ VOTRE CODE

RIGUEUR

**COMPRENDRE TOUT CE
QUE VOUS PRODUISEZ**

À CHAQUE SOUTENANCE

**SAVOIR L'EXPLIQUER
CLAIREMENT**

« TOUT », C'EST

- Le code de vos coéquipiers
- Le code généré par des outils IA
- Chaque choix technique, et pourquoi vous l'avez fait

ATTENDUS

PROJET AUTONOME

Aucune dépendance à télécharger à la main.

UN VRAI WORKFLOW VCS

- Des branches
- Pas de « live-share »
- Des commits explicites, titre et corps

MULTIPLATEFORME

Au moins 2 plateformes supportées.

- Windows/Linux (non, WSL ne compte pas)
- macOS est **aussi** un Unix

ACCESSIBILITÉ

- Prendre en compte les handicaps physiques, audiovisuels et cognitifs
- S'inspirer des fonctionnalités des jeux existants, et y penser dès le début du projet

DOCUMENTATION

+ < Obligatoire /> Une documentation soignée est obligatoire, et elle doit être à jour.

UN README UTILE

- Installation
- Utilisation
- Exemples
- Etc.

DOCUMENTATION DÉVELOPPEUR

- Diagrammes (couches, sous-systèmes, UML, etc.)
- Explication des sous-systèmes
- Tutoriels, wiki, how-to
- Documentation dans le code

QUALITÉ DE LA DOCUMENTATION

ÉCRITE POUR DES HUMAINS

- Un lecteur ne doit jamais buter sur une phrase peu naturelle
- Une doc écrite par IA se repère à ses tournures : réécrivez-la avec vos mots

DE VRAIS SCHÉMAS

- Pas de schéma dessiné en Markdown ou en ASCII
- Un outil adapté : Mermaid s'affiche directement sur GitHub

CONTRÔLE QUALITÉ

- Mettez en place une relecture de la doc, comme pour le code
- Des agents et des skills IA peuvent prendre en charge une partie de ces contrôles

DOC : PROTOCOLE

CE QUI FAIT UNE BONNE DOC DE PROTOCOLE

- Indépendante du langage
- Suffisante pour écrire un client
- Liste les erreurs possibles
- Explique comment les gérer
- Suit un format standard (RFC)

CONSEILS

- Taille des entiers, padding interne, etc. dépendent du compilateur et du langage
- Un enum ne veut rien dire. Ses valeurs, si.
- Un schéma est parfois plus clair que des mots.

ÉTUDE COMPARATIVE

OBLIGATOIRE

**L'ÉTUDE COMPARATIVE EST AUSSI
OBLIGATOIRE**

JUSTIFIER CHAQUE CHOIX

- Langage(s)
- Bibliothèques
- Algorithmes
- Techniques réseau
- Etc.

ÉTUDE COMPARATIVE

PAR OÙ COMMENCER ?

ALGORITHMES ET STRUCTURES DE DONNÉES

- Design patterns
- Algorithmes existants ou sur mesure
- Etc.

QUELLE TECHNIQUE DE STOCKAGE SELON

- Langage(s)
- Bibliothèques
- Algorithmes

SÉCURITÉ

- Principales vulnérabilités de chaque technologie
- Mise en place d'une surveillance de sécurité

POINTS DIVERS

PDF ET WORD NE SONT PAS DES FORMATS DE DOCUMENTATION.

FORMATS DE DOC DANS LE CODE

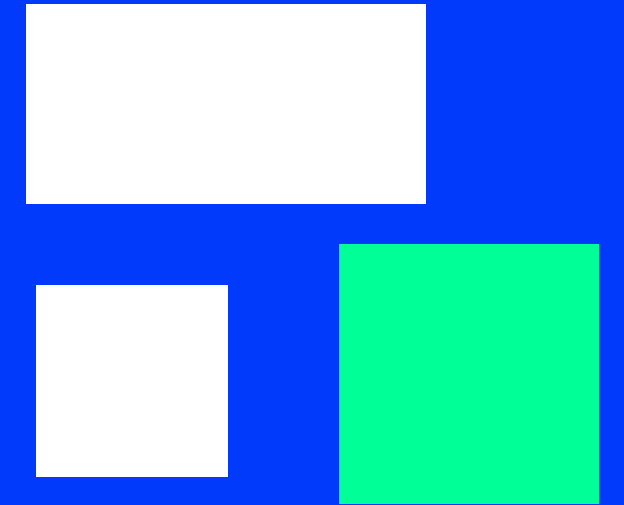
- Intégration à l'IDE
- Génération de doc interactive
- Une bonne pratique en général

DES OUTILS TRÈS UTILISÉS

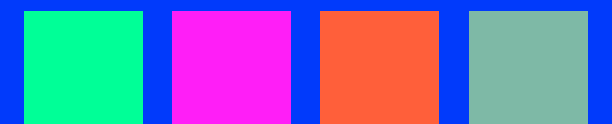
- Markdown
- reStructuredText
- Sphinx
- Doxygen

ASTUCE

Le Markdown de GitHub gère les liens.



PARTIE I : PROTOTYPE



UN PROTOTYPE ?

PAS COMPLET

- Les fonctionnalités de base sont là
- Pas besoin de soigner les détails

DES CHOIX D'ARCHITECTURE RÉFLÉCHIS

Mais gardez en tête que vous refactoriserez plus tard.

LES BONNES QUESTIONS

- Ai-je besoin du réseau dans un jeu multijoueur en ligne ?
- Ai-je besoin d'un menu ?
- Comment mes choix actuels pèsent-ils sur la suite du projet ?

OBJECTIFS

CONCEVOIR VOTRE MOTEUR MULTIJOUEUR EN RÉSEAU

Avec une vraie architecture
logicielle.

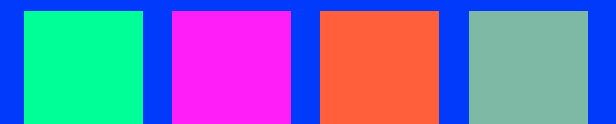
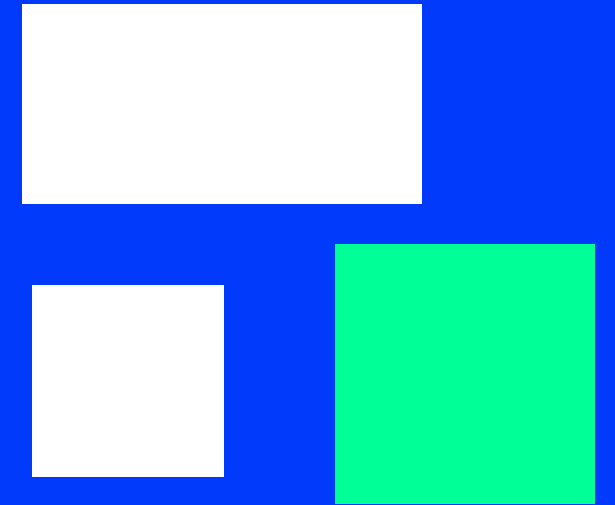
CRÉER UN GAMEPLAY BASIQUE FAÇON R-TYPE

- Un joueur qui se déplace
- Un monstre qui se déplace
- Des interactions de base
(tuer des ennemis, mourir,
etc.)

PARTIE II

Avoir une base solide pour la
partie II.

PARTIE II : AMELIORATIONS



ET MAINTENANT ?

ON NE PEUT QU'AVANCER. REFACTORISER, C'EST PROGRESSER.

3 TRACKS TECHNIQUES

- Moteur de jeu avancé
- Réseau avancé
- Game design avancé

PLUSIEURS SUJETS PAR TRACK

- Vous pouvez piocher des sujets dans chaque track
- Qualité et difficulté > quantité
- Priorité à la pérennité du code

PROCHAIN KICK-OFF

Plus d'informations au prochain kick-off.

VISER L'EXCELLENCE

POUSSEZ LE PROJET AU MAXIMUM, PUIS PROUVEZ-LE AVEC UN ÉVÉNEMENT.

TRACK MOTEUR DE JEU

Organisez une game jam sur le campus.

- Chaque jeu doit utiliser votre moteur
- **Réussite** : au moins 2 jeux faits par des personnes extérieures au groupe
- Ces 2 jeux ne doivent pas être du même genre que le ou les vôtres

TRACK RÉSEAU

Organisez une LAN.

- 20 joueurs minimum, idéalement certains à distance : ailleurs en France, voire à l'étranger (pourquoi pas les Tek4 ?)
- **Réussite** : 20 joueurs en même temps sur toute une session, sans crash du serveur, latence et pertes de paquets mesurées

TRACK GAME DESIGN

Organisez une compétition : speedrun, meilleur score, ou affrontement (1v1 ou équipe contre équipe).

- Des joueurs extérieurs au groupe
- **Réussite** : un classement ou un tableau de tournoi vérifiable par tous : replays ou validation côté serveur

QUAND

Avant ou pendant la semaine de la soutenance finale.

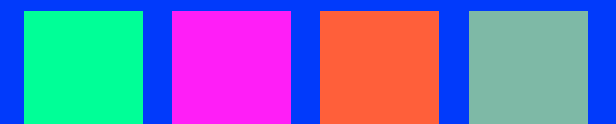
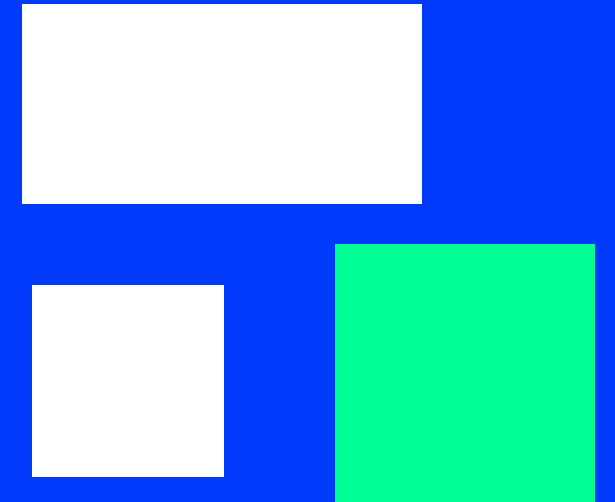
CONDITION

Le projet doit être validé à la 1^{re} soutenance (décision jusqu'à une semaine après). Un bon projet (autour du grade A) ne suffit pas.

RÉCOMPENSE

Les groupes qui y parviennent seront récompensés.

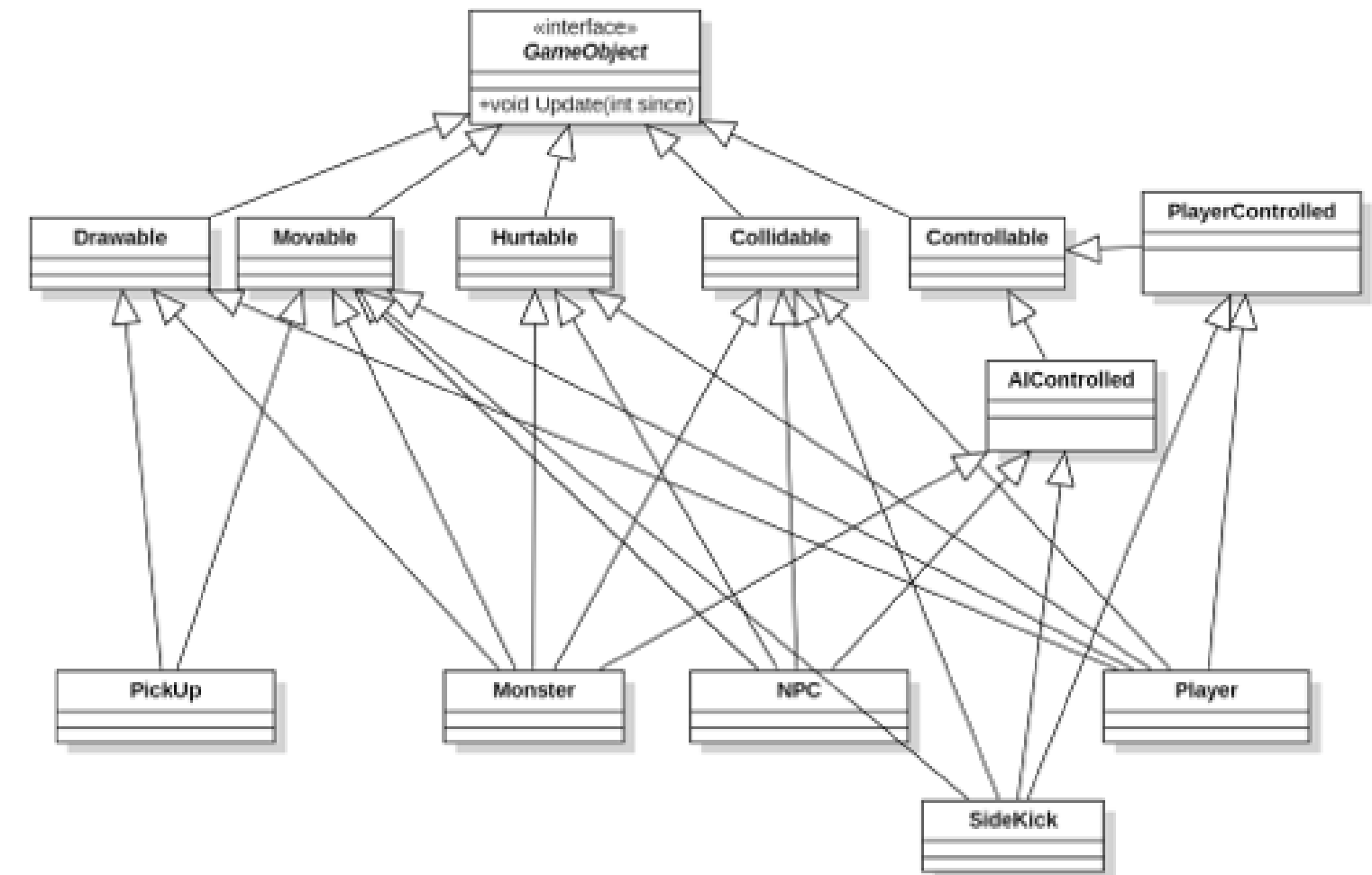
DESIGN, CONCEPTS & IDEEES



ARCHITECTURE MOTEUR : POO

QUESTIONS

- Des objets sans rapport doivent-ils avoir un ancêtre commun ?
- Combien d'ancêtres un objet devrait-il avoir ? Est-ce que ça a du sens ?
- Comment gérer les changements dynamiques ?
- Comment gérer la configuration ?
- Peut-on faire mieux ? **La composition plutôt que l'héritage !**



ARCHITECTURE MOTEUR : ECS

ENTITY

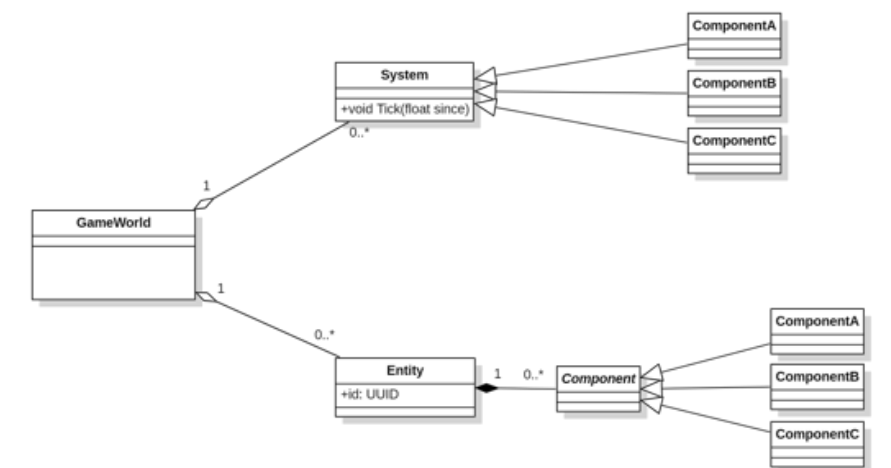
- Identifie une « chose » dans le moteur
- Peut n'être qu'un simple ID

COMPONENT

- Identifie un trait, une capacité d'une entité
- Simple POCO (struct)
- Contient des données

SYSTEM

- Implémente la logique et les comportements
- De simples fonctions (ou scripts)
- La plupart ne travaillent que sur quelques composants
- Applique la même fonction à toutes les entités qui ont les composants voulus



DESIGN MULTIJOUEUR

QUE DOIT FAIRE LE SERVEUR ?

TOUT ?

- Peut garantir la cohérence des données
- En général, une mauvaise idée
- Le réseau, c'est leeeeent

« SEULEMENT » SALONS, CHAT ET MATCHMAKING ?

- Qui est responsable de la cohérence des données ?



DESIGN MULTIJOUEUR

OÙ SE TROUVE LE MOTEUR ?

CÔTÉ SERVEUR UNIQUEMENT ?

- + Client plus simple
- + Rendu et gestion des entrées seulement
- + Client perdu sans flux de données constant
- Le serveur doit gérer les entrées

CÔTÉ CLIENT UNIQUEMENT ?

- + Un client doit garantir la cohérence
- + Chaque client fait le rendu de sa partie
- L'hôte peut tricher
- L'hôte doit être assez puissant pour 2 parties

SERVEUR ET CLIENT

- + Le serveur est responsable des données
- + Le client réagit « instantanément » aux entrées (contrôles plus serrés)
- Il faut une forme de réconciliation (rollback ?)
- Moins de calcul au total (le serveur ignore les entrées)

RÉSEAU & JEUX

QUESTIONS À SE POSER

- Qu'est-ce que j'envoie ?
- À quelle fréquence ?
- Ai-je besoin de vitesse ou de fiabilité ?
- Que se passe-t-il si un message n'arrive jamais ? S'il arrive deux fois ?

QUELQUES RÉPONSES

- Les entrées utilisateur **SONT** importantes
- Les données de jeu... beaucoup moins (vous pouvez en perdre une partie)

RÉSEAU : PROTOCOLE

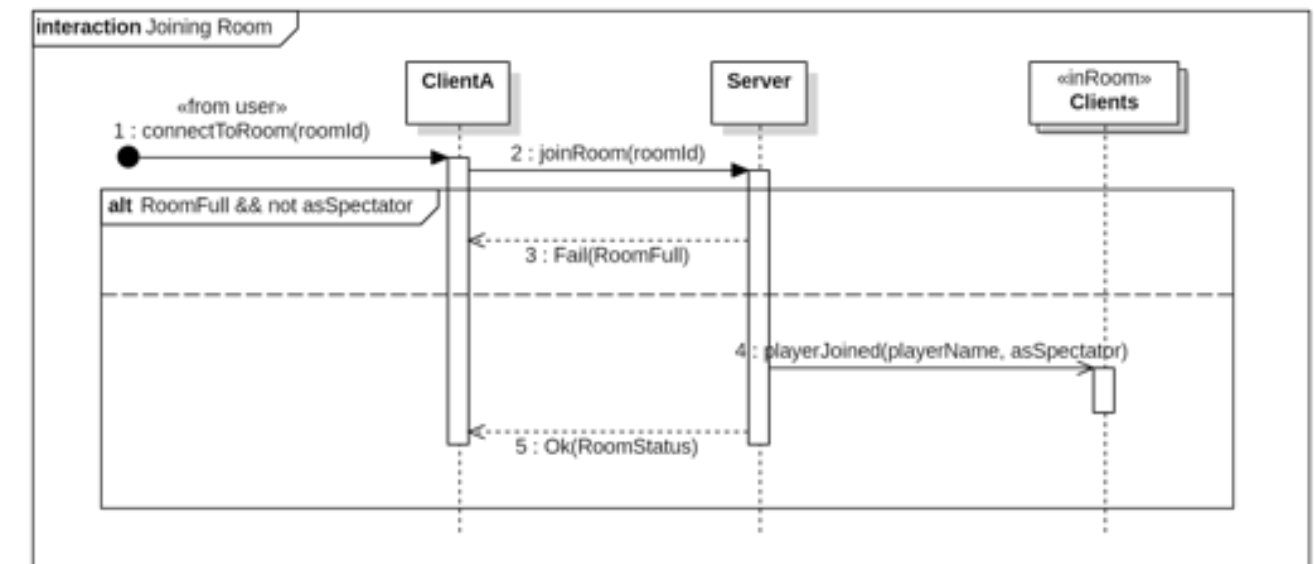
TEXTE OU BINAIRE ?

- Le texte est lisible par un humain
- Le binaire est plus court, plus rapide et n'a pas besoin de parsing

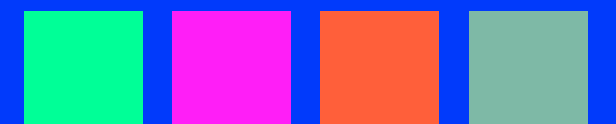
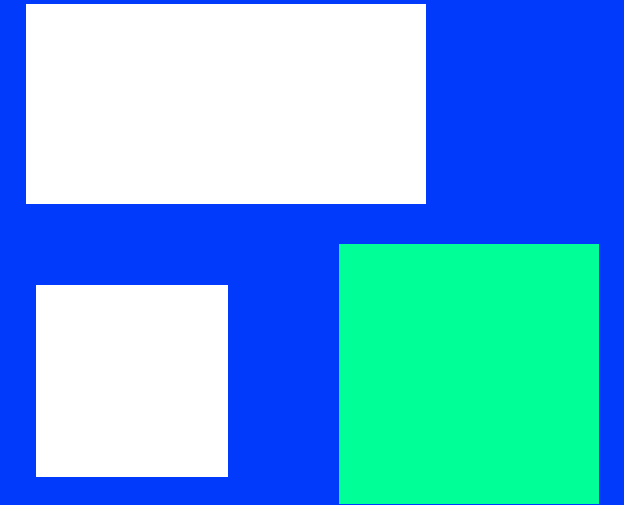
**N'ENVOYER QUE
LES DONNÉES
NÉCESSAIRES**

MODÉLISER VOTRE PROTOCOLE

- Facilite la programmation
- Une information visuelle se comprend plus facilement



RESSOURCES



LES OUTILS DU MÉTIER

GESTION DES DÉPENDANCES

- Git submodules
- CMake (FetchContent)
- Gestionnaires de paquets (Conan, vcpkg)

CI/CD

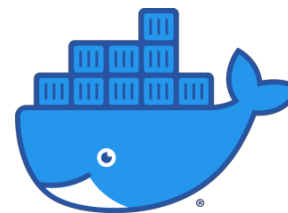
- Windows/Linux (non, WSL ne compte pas)
- macOS est **aussi** un Unix

BONNES PRATIQUES ET GARDE-FOUS

- Frameworks de test (Catch2, Criterion, GoogleTest, etc.)
- Linter et formatter (clang-tidy, etc.)

DOCUMENTATION

- Dia, StarUML, Lucidchart
- Doxygen, Read the Docs, Sphinx, etc.



LES OUTILS DU MÉTIER

BUILD SYSTEMS

- **CMake** le standard de fait
- **Xmake** Lua, gestionnaire de paquets intégré
- **Meson** rapide, génère des fichiers Ninja
- **Premake** Lua, génère des projets d'IDE
- **Bazel** Google, builds reproductibles
- **build2** toolchain C++ avec paquets
- **SCons** scripts de build en Python
- **Ninja** le backend sous CMake et Meson

Les compilateurs derrière : GCC, Clang, MSVC.

MOTEURS ET BIBLIOTHÈQUES ECS À ÉTUDIER

- **Bevy** Rust, ECS de bout en bout
- **Engine²** C++, cœur ECS, chaque fonctionnalité est un plugin
- **Unity DOTS** la stack data-oriented de Unity : ECS, Jobs, Burst
- **Godot** open source, arbre de scènes fait de nœuds
- **O3DE** C++, open source, « Gems » modulaires
- **Defold** cœur C++, gameplay scripté en Lua
- **A-Frame** web, ECS pour WebXR, déclaré en HTML sur three.js
- **Unreal Engine** C++, acteurs et composants à l'échelle production
- **Nazara Engine** framework C++ multiplateforme : 2D/3D, audio, réseau
- **Spartan Engine** moteur C++ de R&D, rendu piloté par le GPU
- **Hazel** la série YouTube de The Chernobyl : un moteur construit épisode par épisode
- **Wicked Engine** C++, open source, rendu temps réel moderne
- **id Tech 3** source de Quake III Arena (GPL), une référence du netcode client/serveur
- **EnTT** bibliothèque ECS C++ header-only, utilisée dans Minecraft

QUESTIONS ?

G5 · Advanced C++